Prompt:

The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.

• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.

• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)

• Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).

• This approach, by the way, is called depth-first search with backtracking. For

each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.

• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.

• Write a recursive object-oriented program in C++ to find the solution to the five puzzle files.

• The output for each solution, should include:

o The puzzle file name (e.g., puzzle1.txt)

o The puzzle from the file printed out.

o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.

• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.

Puzzle1.txt looks like:

5 _ _ _ _ _ 1 7 _

1 _ 6 5 _ 9 _ 4 _

4 7 2 1 _ 6 _ _ _

9 _ _ _ _ _ 5 _ _

_ 1 8 _ 9 5 4 _ _

6 _ _ 4 _ 2 3 8 9

_ 4 _ _ _ _ 9 3 _

_ 9 _ 7 _ 3 _ 5 _

2 6 3 9 5 8 7 1 4

The output should look like this:

5 3 9 8 2 4 1 7 6

1 8 6 5 7 9 2 4 3

4 7 2 1 3 6 8 9 5

9 2 4 3 8 7 5 6 1

3 1 8 6 9 5 4 2 7

6 5 7 4 1 2 3 8 9

7 4 5 2 6 1 9 3 8

8 9 1 7 4 3 6 5 2

2 6 3 9 5 8 7 1 4

Solutions found: 1

Claude.ai (accessed from web):

#include <iostream>

#include <fstream>

#include <vector>

#include <string>

using namespace std;

class Sudoku { private: static const int SIZE = 9; string fileName; int grid[SIZE][SIZE]; // 0 = blank vector<vector<vector>> solutions;

```
// Can value v go at (row, col) without breaking row/column/block
rules?
bool isCandidate(int row, int col, int v) const {
    for (int i = 0; i < SIZE; i++) {
        if (grid[row][i] == v || grid[i][col] == v) return false;
    }
    int br = (row / 3) * 3, bc = (col / 3) * 3;
    for (int r = br; r < br + 3; r++)
        for (int c = bc; c < bc + 3; c++)
            if (grid[r][c] == v) return false;
    return true;
}

// Recursive depth-first search with backtracking; records every
solution.
void solve() {
    // Find the next empty cell
    int row = -1, col = -1;
    for (int r = 0; r < SIZE && row == -1; r++) {
        for (int c = 0; c < SIZE; c++) {
            if (grid[r][c] == 0) { row = r; col = c; break; }
        }
    }

    // Base case: no empty cells left (k == 81) -> save a copy of the
grid
```

```
    if (row == -1) {
        vector<vector<int>> copy(SIZE, vector<int>(SIZE));
        for (int r = 0; r < SIZE; r++)
            for (int c = 0; c < SIZE; c++)
                copy[r][c] = grid[r][c];
        solutions.push_back(copy);
        return;
    }

    // Recursive case: try each candidate, then undo it (backtrack)
    for (int v = 1; v <= SIZE; v++) {
        if (isCandidate(row, col, v)) {
            grid[row][col] = v;
solve();

                grid[row][col] = 0;

            }

        }

    }



    template <typename Grid>

    static void print(const Grid& g) {

        for (int r = 0; r < SIZE; r++) {

            for (int c = 0; c < SIZE; c++) {

                if (g[r][c] == 0) cout << '_';

                else cout << g[r][c];

                if (c < SIZE - 1) cout << ' ';

            }

            cout << '\n';
```

```
        }
    }

public:
    explicit Sudoku(const string& file) : fileName(file) {}

    bool load() {
        ifstream in(fileName);
        if (!in) return false;
        for (int r = 0; r < SIZE; r++) {
            for (int c = 0; c < SIZE; c++) {
                string token;
                if (!(in >> token)) return false;
                grid[r][c] = (token == "_") ? 0 : stoi(token);
            }
        }
        return true;
    }

    void run() {
        cout << fileName << '\n';
        cout << "Puzzle:\n";
        print(grid);
```

```
        cout << '\n';


        solve();


        if (solutions.empty()) {
            cout << "No solution found\n";
        } else {
            int n = 1;
            for (const auto& s : solutions) {
                if (solutions.size() > 1) cout << "Solution " << n++
<< ":\n";
                print(s);
                cout << '\n';
            }
            cout << "Solutions found: " << solutions.size() << '\n';
        }
        cout << "----------------------------------------\n";
    }
};


int main() {
    for (int i = 1; i <= 5; i++) {
        string name = "puzzle" + to_string(i) + ".txt";
```

```
        Sudoku puzzle(name);
        if (!puzzle.load()) {
            cout << name << ": could not be read\n";
            continue;
        }
        puzzle.run();
    }
    return 0;
}
```

Gemini:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>


class SudokuBoard {
private:
    int grid[9][9];


    // Check if placing 'num' at grid[row][col] is valid
    bool isValid(int row, int col, int num) const {
        // Check row
        for (int c = 0; c < 9; ++c) {
            if (grid[row][c] == num) return false;
        }
        // Check column
        for (int r = 0; r < 9; ++r) {
            if (grid[r][col] == num) return false;
        }
        // Check 3x3 subgrid
        int startRow = row - row % 3;
```

```
    int startCol = col - col % 3;
    for (int r = 0; r < 3; ++r) {
        for (int c = 0; c < 3; ++c) {
            if (grid[startRow + r][startCol + c] == num) return false;
        }
    }
    return true;
}

// Find the next empty cell (represented by 0)
bool findEmptyCell(int &row, int &col) const {
    for (row = 0; row < 9; ++row) {
        for (col = 0; col < 9; ++col) {
            if (grid[row][col] == 0) {
                return true;
            }
        }
    }
    return false;
}

// Recursive backtracking helper function
```

```
void solveRecursive(int &solutionCount, std::vector<std::vector<int>> &solutions) {
    int row, col;
    if (!findEmptyCell(row, col)) {
        // Puzzle solved successfully; store the configuration
        std::vector<int> currentSolution;
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                currentSolution.push_back(grid[r][c]);
            }
        }
        solutions.push_back(currentSolution);
        solutionCount++;
        return;
    }

    // Try candidates 1 through 9
    for (int num = 1; num <= 9; ++num) {
        if (isValid(row, col, num)) {
            grid[row][col] = num;
            solveRecursive(solutionCount, solutions);
            grid[row][col] = 0; // Backtrack
        }
```

```
        }
    }

public:
    SudokuBoard() {
        for (int i = 0; i < 9; ++i)
            for (int j = 0; j < 9; ++j)
                grid[i][j] = 0;
    }

    // Load puzzle from file where '_' represents blank squares
    bool loadFromFile(const std::string &filename) {
        std::ifstream file(filename);
        if (!file.is_open()) return false;

        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                char ch;
                if (!(file >> ch)) return false;
                if (ch == '_') {
                    grid[r][c] = 0;
                } else if (ch >= '1' && ch <= '9') {
```

```
            grid[r][c] = ch - '0';
        } else {
            return false;
        }
      }
    }
    return true;
}

// Print the puzzle grid
void print() const {
    for (int r = 0; r < 9; ++r) {
      for (int c = 0; c < 9; ++c) {
        if (grid[r][c] == 0) {
            std::cout << "_ ";
        } else {
            std::cout << grid[r][c] << (c == 8 ? "" : " ");
        }
      }
      std::cout << "\n";
    }
}
```

```
// Solve the puzzle and print results in the required format
void solve(const std::string &filename) {
    std::cout << filename << "\n";

    int solutionCount = 0;
    std::vector<std::vector<int>> solutions;
    solveRecursive(solutionCount, solutions);

    if (solutionCount == 0) {
        std::cout << "No solution found\n";
    } else {
        for (int i = 0; i < solutionCount; ++i) {
            int idx = 0;
            for (int r = 0; r < 9; ++r) {
                for (int c = 0; c < 9; ++c) {
                    std::cout << solutions[i][idx++] << (c == 8 ? "" : " ");
                }
                std::cout << "\n";
            }
        }
    }
```

```
        std::cout << "Solutions found: " << solutionCount << "\n\n";
    }
};


int main() {
    for (int i = 1; i <= 5; ++i) {
        std::string filename = "puzzle" + std::to_string(i) + ".txt";
        SudokuBoard board;
        if (board.loadFromFile(filename)) {
            board.solve(filename);
        } else {
            std::cout << filename << "\nCould not open or read file.\n\n";
        }
    }
    return 0;
}
```

Analysis:

Correctness:

Claude:

Code works and handles edge cases. Handles Errors.

Gemini:

Code works but doesn't have correct output format, handles edge cases. Handles Errors.

Time complexity:

Claude:

Runtime of .013 seconds

Multiple nested for loops and uses recursion.

Gemini:

Runtime of .006 seconds

Multiple nested for loops and uses recursion.

Space Complexity:

Claude:

Used a maximum of 3840 kB of ram.

Gemini:

Used a maximum of 896 kB of ram.

Maintainability:

Claude:

Very limited comments explaining the bare minimum of each function

Gemini:

Similar level of comments.

Selection:

Gemini, although the code didn't have the correct output format, that is not hard to change. It had faster and more space efficient code and the same maintainability.

Changes:

Maintainability:

I will add both a prologue comment and comments on every line of code explaining what they do.

Correctness:

I will fix the output to print the puzzle for each file and add spaces between solutions.